# System Level Security Measures Implemented in OpenBSD

**Anirudha Bose**
Dept of Computer Science & Engineering
IIIT Bhubaneswar
ani07nov@gmail.com

# OpenBSD

- UNIX-like operating system derived from BSD 4.4
- Forked from NetBSD in 1995
- Ulta-secure and elegant in design
- Most secure operating system in the world
- Kernel + userland + documentation maintained together
- Third party applications available via the **ports** system
- New release every six months

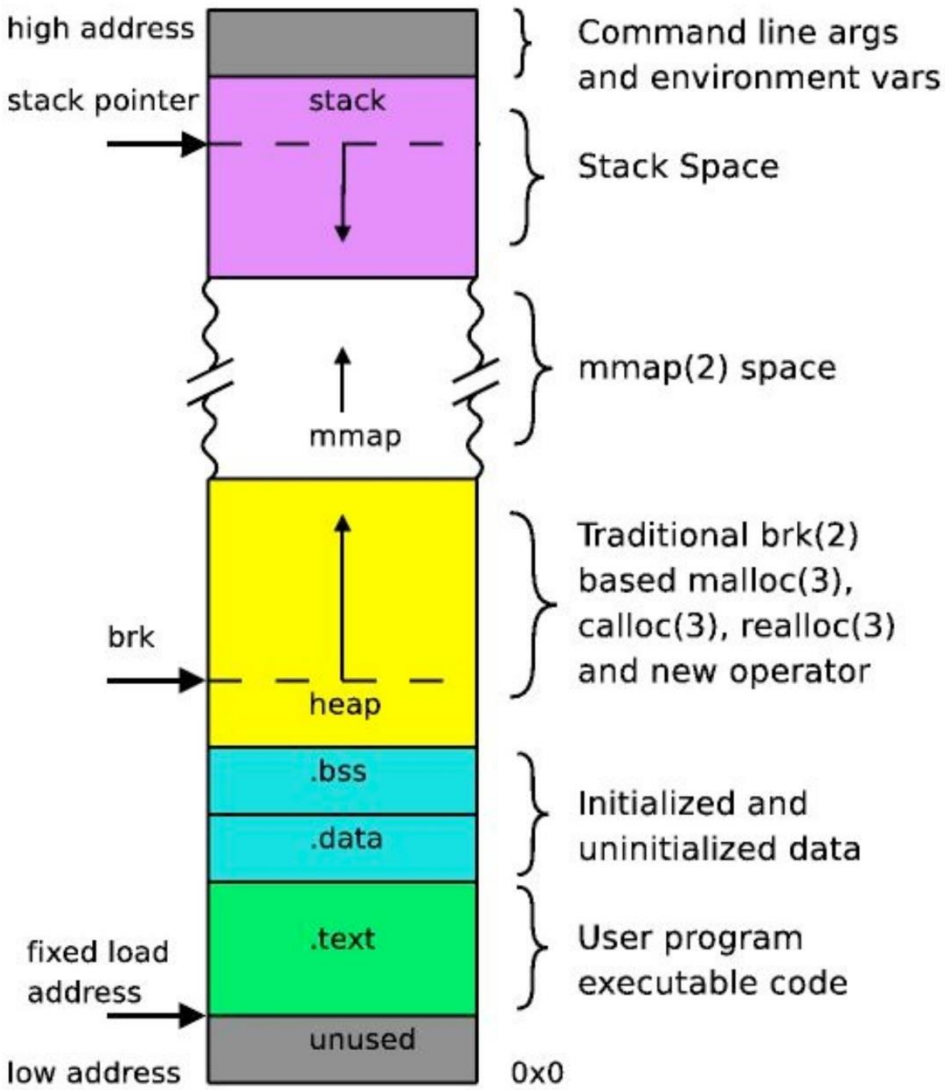
# Goals and Objectives

- ❑ Emphasis on code correctness, reliability, security, standardization, and portability
- ❑ Peer review of patches at every level
- ❑ Distribute open source code under free and liberal BSD licenses
- ❑ Compliance with standards like ANSI and POSIX
- ❑ Machine independent source, and cross-platform reliability
- ❑ Supported hardware architectures: i386, alpha, AMD64, ARM, macppc, SPARC, SPARC64, sgi, VAX, socppc, octeon, HPPA, landisk
- ❑ Provide a good set of security tools for SSH, SSL, IPSEC, etc like OpenSSH and LibreSSL
- ❑ Detect and fix security issues before anyone else

# System Level Security Technologies: Overview

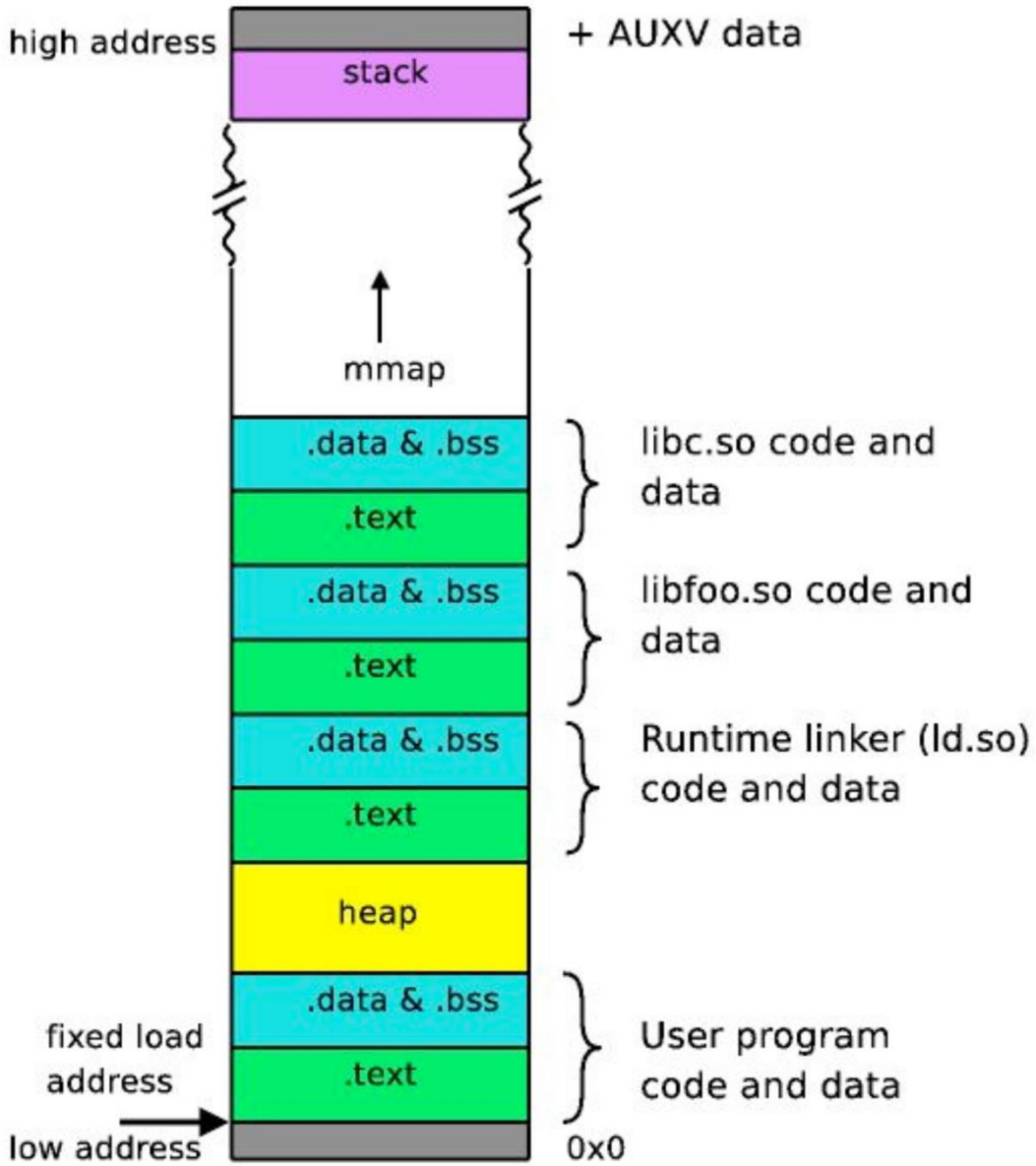
- ❑ ASLR with support for Position Independent Executables (PIE)
- ❑ W^X Policy
- ❑ OpenBSD StackGap
- ❑ Stack Smashing Protection (SSP) / ProPolice
- ❑ GCC `-Wbounded` and `__bounded__` attribute
- ❑ The `.rodata` segment
- ❑ Use of randomness (`ld.so, malloc(3), mmap(2)`)
- ❑ Safe string handling using `strlcpy()` and `strlcat()`
- ❑ Swap encrypted in small pieces, each with separate keys. Keys frequently destroyed

# Traditional Process Address Space: Static Program



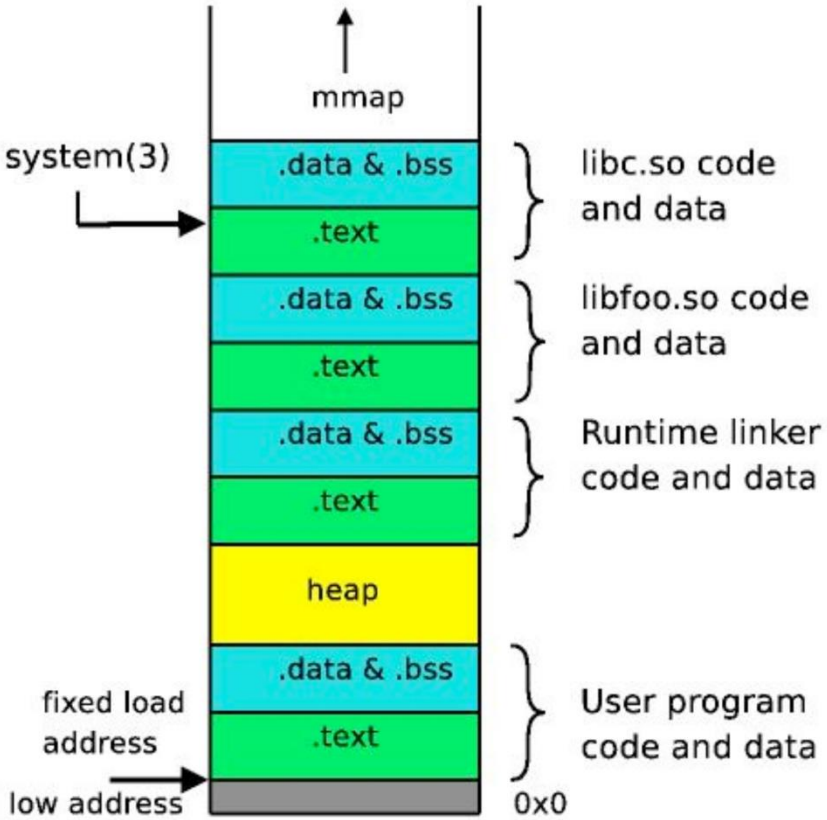

- ❑ Kernel mmaps `.text` and `.data` from executable
- ❑ Allocates `.bss` space and initializes to zero
- ❑ Reserves head address space – `brk(2)` based `malloc(3)`
- ❑ Reserves stack address space – stack grows down
- ❑ Bottom of stack initialized with environment variables and commandline arguments
- ❑ Space in between stack and heap utilized for `mmap(2)`

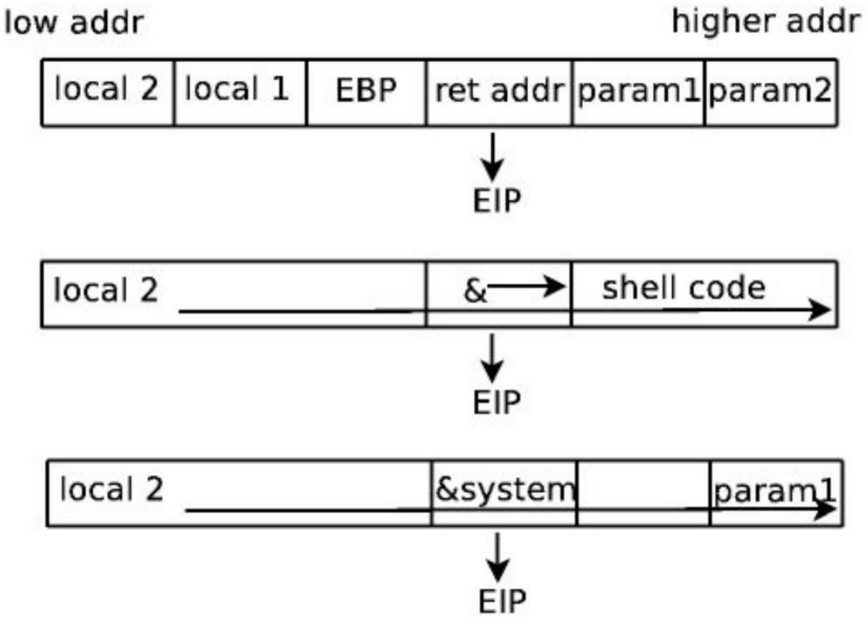
# Traditional Process Address Space: Program with Shared Libs




- Similar to static program
- Kernel also mmaps runtime linker (`ld.so`) and adds auxiliary vector data to bottom of stack
- `ld.so` is executed first which mmaps the rest of the shared libraries in the order they were linked
- Each invocation of user program has same process layout
- Only changing the data size hard limit will change location of shared libraries.

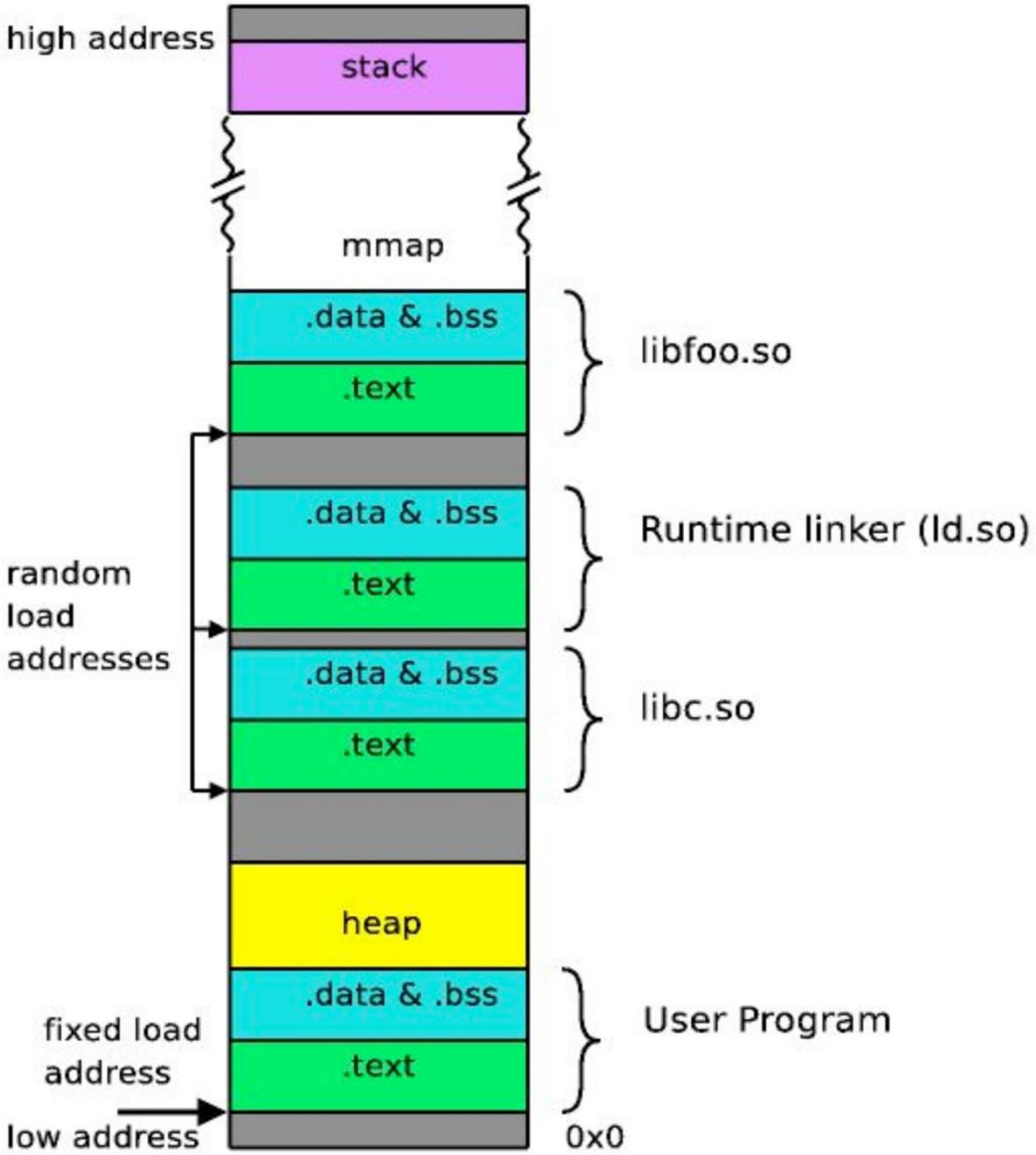
# Return-To-Libc Attack



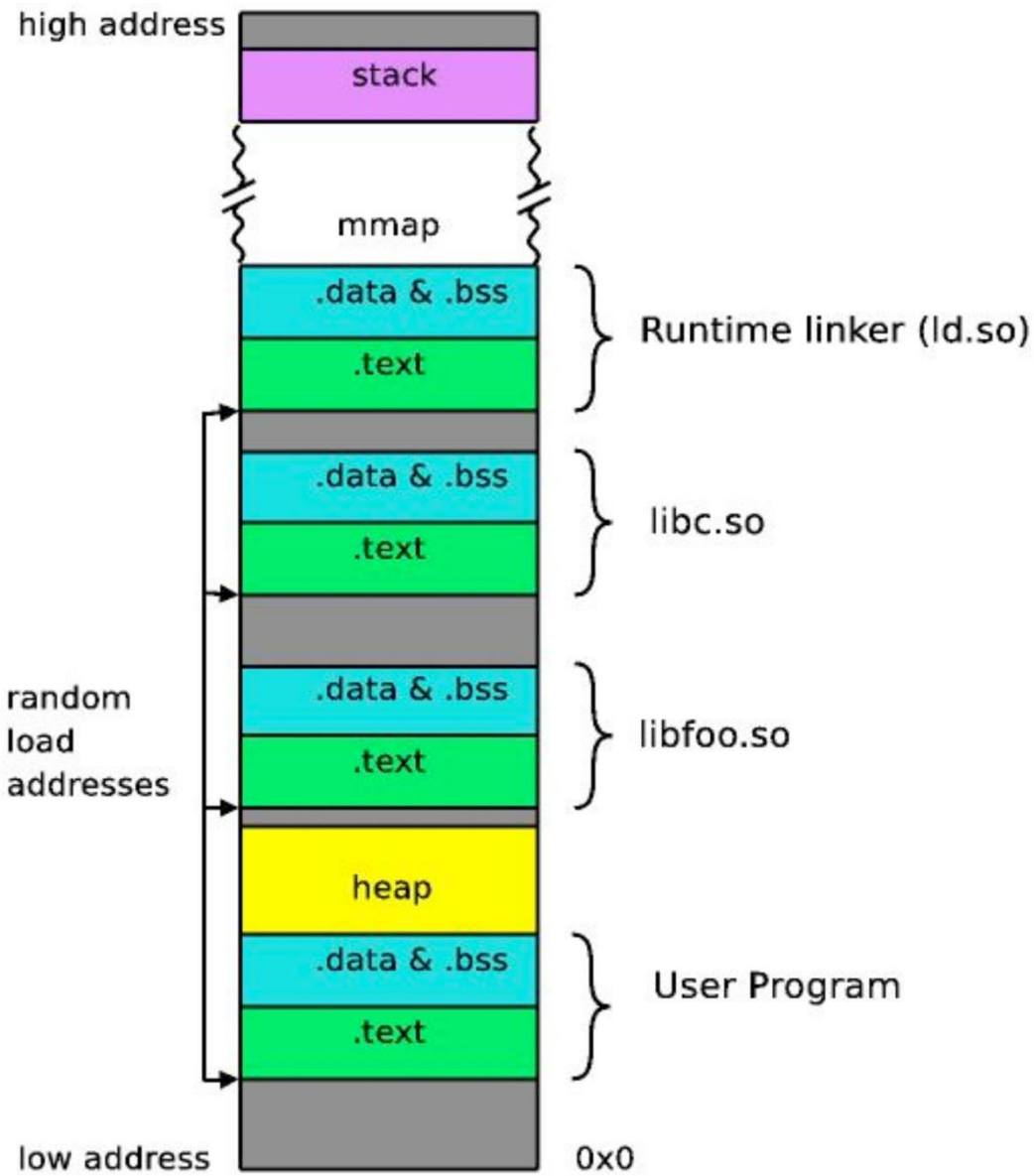

- Variation of classic buffer overflow attack
- Avoids the need for an executable stack
- Takes advantage of predictable addresses.

# OpenBSD Address Space Layout: without PIE




- ❑ Ordinary Address Space Layout Randomization (ASLR)
- ❑ Executable at fixed load address
- ❑ Randomized `mmap(2)`
- ❑ Shared libraries with random load address and order
- ❑ Main program `.text` is still vulnerable.

# OpenBSD Address Space Layout: with PIE Support

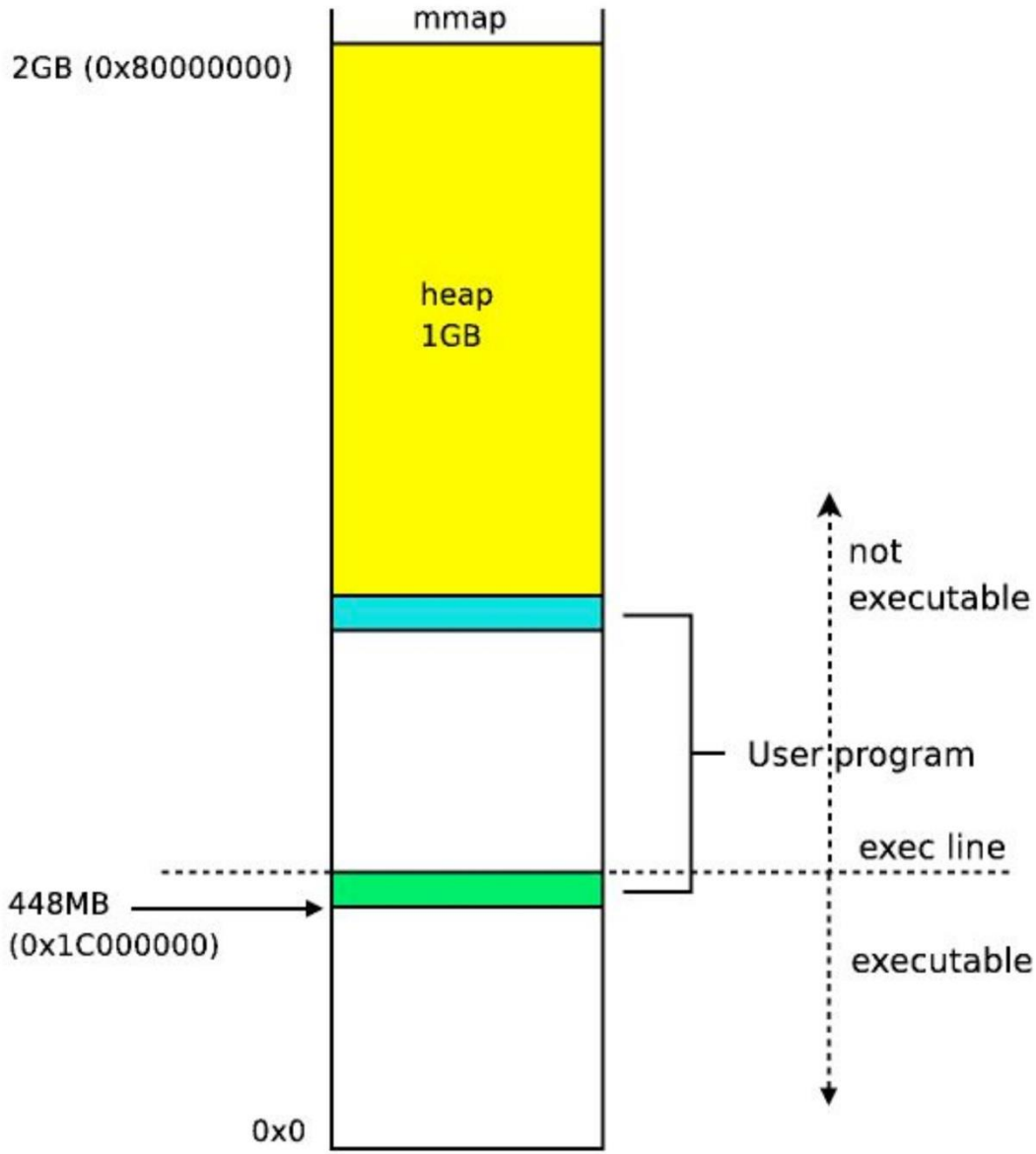



- ❑ Improved Address Space Layout Randomization (ASLR)
- ❑ Main program `.text` no longer has fixed load address
- ❑ Must compile position independent (less efficient) code
- ❑ `mmap(2)` space shrinks to make room
- ❑ Shared libraries still have random load address and order

# W^X Policy

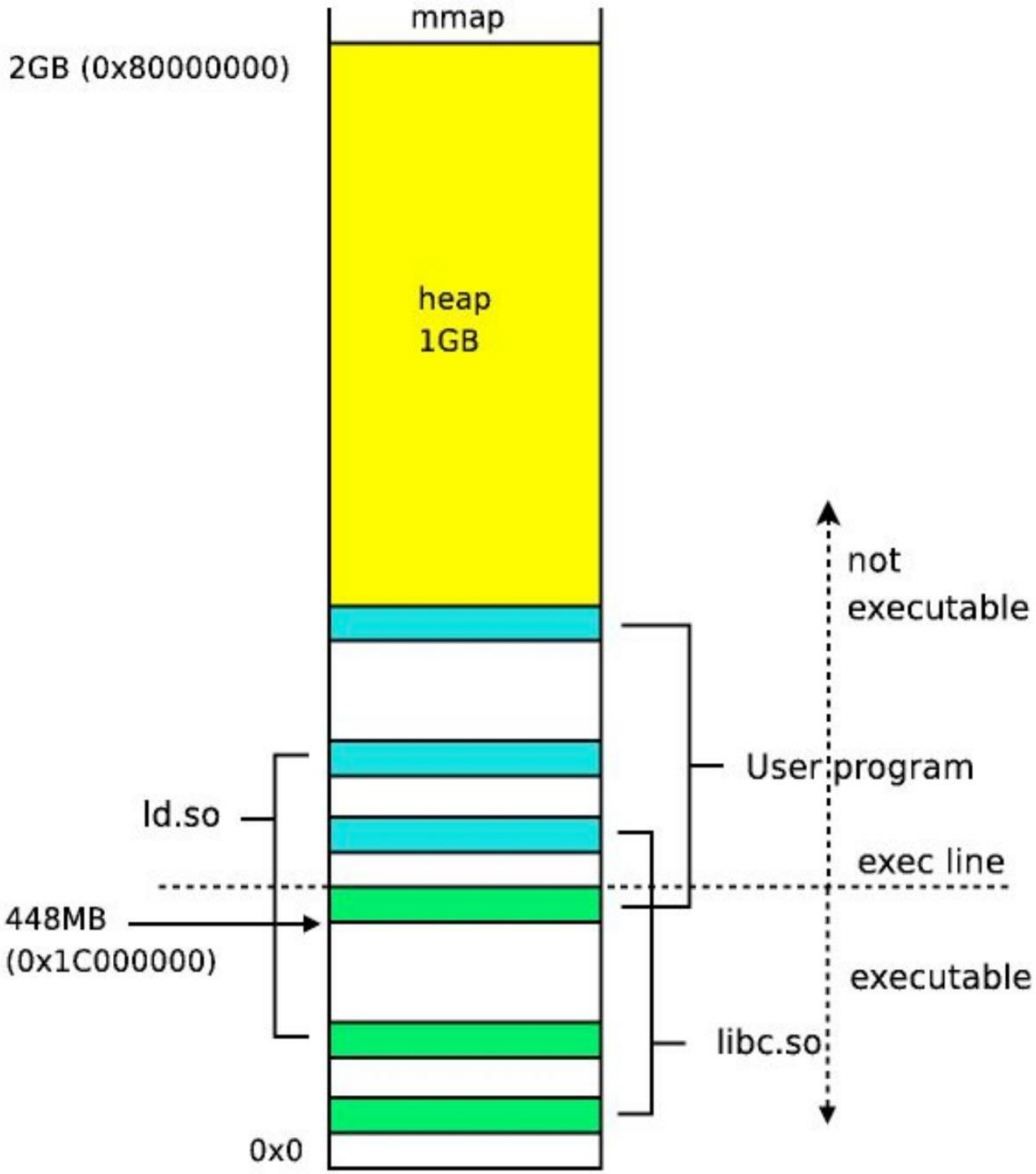
- Many exploits rely on the fact that address space has memory which is both writable as well as executable (W and X)
- Need to have a generic policy for the whole address space
- W^X policy: A page may be both writable or executable, but not both. Hence W^X.
- Easy to implement on architectures with fine grained page permissions (per-page X bit) like X86_64, sparc, sparc64, ARM, etc.
- Harder on other architectures like i386 and powerpc
- Impossible on architectures like VAX, mips, and m68k

# Intel i386 W^X




- ❑ No NX bit in page tables
- ❑ Range of execution – exec line 512 MB
- ❑ A gap of 512MB between start of `.text` and start of `.data`
- ❑ User program loaded high at a fixed address and heap follows the `.data` segment
- ❑ Leaves a gap below exec line for the executable pages

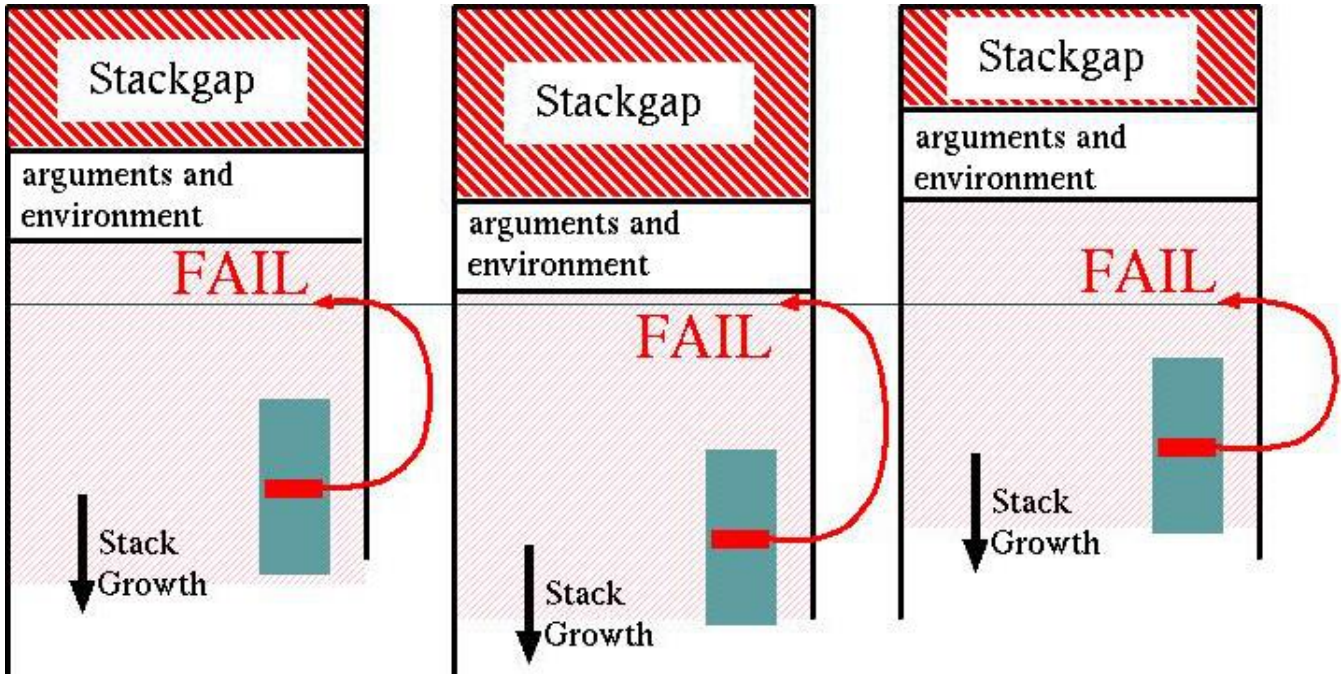
# Intel i386 W^X (contd.)



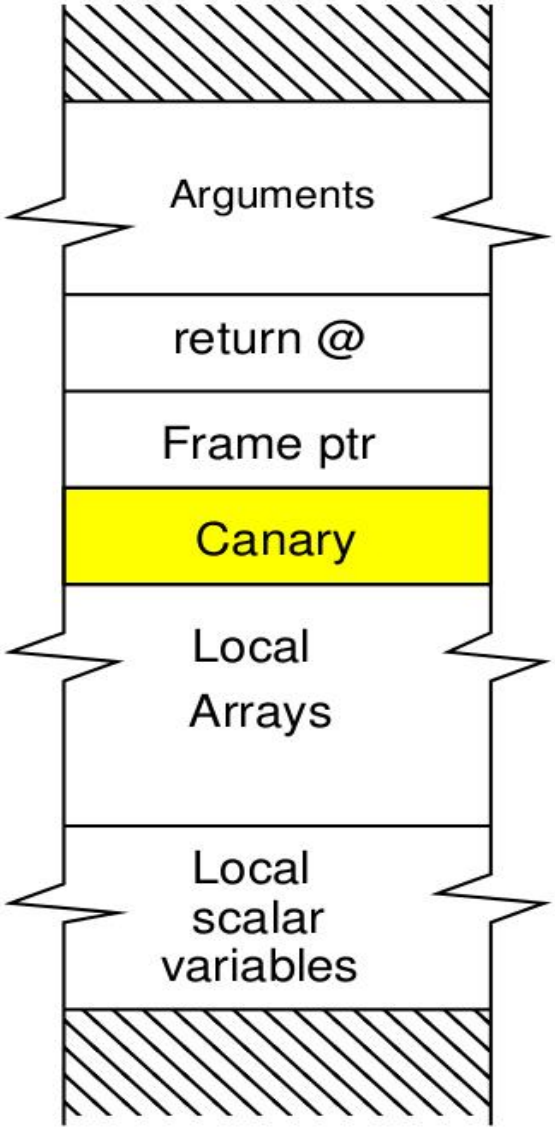

- ❑ `ld.so` and shared libraries `.text` loaded into random addresses below exec line
- ❑ `.data` loaded at 512MB offset from `.text`
- ❑ `.data` segment is guaranteed to be non-executable

# OpenBSD StackGap

- Stack overflows are the easiest exploitable vulnerability. Attacker overflows the buffer on stack overwriting the function return address, jumping to the malicious shellcode.
- Range of the random stack gap can be controlled with the `kern.stackgap_random` sysctl. A value of 0 disables it. The default value is 64K.
- Solution: keep a random sized gap at the top of stack
- Wastes at most 1 page of real memory



# ProPolice




- **Principle:** put a "canary" on the stack, in front of local variables

- Prologue inserts a random value, the canary, on the stack before local arrays.
- Function epilogue checks the value before return

- If still alive (unchanged) – no buffer overflow

- If dead (changed) – possible overflow; call `abort()`

- GCC + Propolice (SSP) activated by default to build all libraries and applications.

# GCC –`Wbounded` option and `__bounded__` attribute

- A compiler option that performs static checking to make sure the bounds length passed to common functions matches the real buffer length.

- Functions must be marked with the new attribute `__bounded__`, and warnings are turned on by `-Wbounded`.

```
char buf[1024]; /* buf size fixed at compile time */
snprintf(buf, SIZE, fmt); /*-Wbounded will check if SIZE==1024*/
```

- Original patch by OpenBSD, now merged upstream (GCC). Present in all modern compilers.

# The `.rodata` segment

- Read-only strings and pointers were stored in the `.text` segment giving them the attributes `PROT_READ` and `PROT_EXEC`

- Const data could be executed by a potential attacker

- Idea is to create a new `.rodata` segment with `PROT_READ` attribute only thus loses `PROT_EXEC` attribute

- Ensures that objects in the address space have a minimal set of permissions

# Random Numbers in OpenBSD

A good random numbers source with accurate entropy measure is important for security.

**Entropy generation:**

- ❑ from I/O devices like keyboard, mouse, network or audio cards
- ❑ from hardware sources like CPU and dedicated crypto chips

Centralized random number generation system in OpenBSD improves security since it is harder to observe or predict

**`arc4random()`:**

- ❑ OpenBSD's own random number generator to provide higher quality data than those described in `rand(3), random(3), and rand48(3).`
- ❑ Use of these functions is encouraged for almost all random number consumption
- ❑ Other interfaces are deficient in either quality, portability, standardization, or availability.

# Randomness in runtime linker: `ld.so`

- OpenBSD's `ld.so` loads libraries randomly in memory to randomize the shared library order
- Map shared libraries in a different order on every load
- Attempt to map shared libraries randomly into the address space by skewing their base address
- Improved security at the cost of a tiny bit of virtual address space
- Return-to-libc attacks are a lot more difficult
- Much needed feature for programs which load many libraries at runtime, like sshd, and Mozilla services.

# Randomness in `mmap(2)`

- POSIX standard says that if **`MAP_FIXED`** is specified, then it is mandatory to map into that address.

- If however, **`MAP_FIXED`** is not specified, the operating system is free to select any memory location. Other operating systems do this very predictably. OpenBSD adds a security mechanism by randomizing this address.

- Each time the program is run, there is a different address space behavior.

- Not possible to guess the location of a function or code placed in the virtual memory.

# Randomness in `malloc(3)`

- Did you addresses of objects allocated by `malloc(3)` are fairly predictable? Many recent heap attacks have relied on this vulnerability.

- Two cases of allocations by `malloc(3)`:

  Smaller than **`PAGE_SIZE`**:

  * `malloc(3)` maintains buckets of "chunks"
  * randomize chunk selection from the buckets
  * activated by **G** option in `/etc/malloc.conf`
  * Guard pages mechanism: Allocations followed by a guard page; segmentation fault if overflow

  Greater than or equal to **`PAGE_SIZE`**:

  * rely on random `mmap(2)`

- Result: heap attacks more difficult

# Safe string handling with `strlcpy()` and `strlcat()`

- Standard C string functions `strcpy()` and `strcat()` prone to buffer overflow attacks

- More intuitive API compared to the length-bounded `strncpy()` and `strncat()` functions

- The new `strlcpy()` and `strlcat()` API allows safe string copies

- New API guarantees that strings are `NUL` terminated and take the size of string in bytes as parameter

- Does not zero-fill the unused bytes in the new string, unlike `strncpy()` and `strncat()`

- Easy way to detect truncation

# Conclusion

- Features we discussed:
  - * Memory protection
  - * Randomization techniques
  - * Compiler features

- Features not discussed:
  - * Network level security techniques in OpenBSD

- What OpenBSD lacks:
  - * User application protection (web browsers and email clients)
  - * Robust Privilege Separation / Sandboxing
  - * Kernel level privileged code running in user space (X Server)
  - * Efficient memory protection in non-standard hardware – Stack growing up!
  - * W^X policy on unsupported architectures – VAX, M68k, MIPS.

# Thanks You

# Questions?

**References:**


• Peter Valchev, Using OpenBSD Security Features to Find Software Bugs, ACM Reflections/Projections (2007)

• Todd C. Miller, Theo de Raadt, strlcpy and strlcat consistent, safe, string copy and concatenation, USENIX (1999)

• Kurt Miller, Implementing Position Independent Executables (PIE) on OpenBSD, DCBSDCon (2009)

• Theo de Raadt, Exploit Mitigation Techniques, OpenCON (2005)

• Otto Moerbeek, A new malloc(3) for OpenBSD, EuroBSDCon (2009)

• Matthieu Herrb, Protection measures in OpenBSD, H2K9 (2009)